Claim Denial Prediction — Write-Up

Ensemble Health Partners AI/ML Take-Home Assessment

1. Problem framing

The review team can only manually inspect the top 25% of claims by risk score before submission — everything else goes out unreviewed. That constraint, not overall accuracy, should drive the metric: the operational question is *of all claims that will actually be denied, what fraction land inside that top-25% review window?* I call this the **capture rate at top 25%** and used it as the primary model-selection and threshold criterion, with ROC-AUC / PR-AUC reported as threshold-independent summaries.

Errors are asymmetric: a denial that slips outside the review window (false negative) costs a full rework cycle for the hospital, while a healthy claim flagged for review (false positive) only costs a few minutes of a reviewer's time that the team already budgets for. That argues for maximizing recall of denials within the fixed review budget, rather than optimizing for balanced precision/recall or raw accuracy.

2. Data findings worth sharing with a non-technical manager

- The current model catches the large majority of claims denied for **missing documentation (81%)**, **missing prior authorization (73%)**, and **unverified eligibility (72%)** within the top-25% review window — these are exactly the operational, fixable-before-submission issues the review team exists to catch.
- It catches almost none of the claims denied for **payer policy / medical necessity (0%)** or **coding errors (27%)**. This isn't a modeling shortcoming so much as a data-availability one: none of the pre-submission fields speak to clinical necessity or code accuracy, so the model has no signal to work with there. Closing this gap would need a different data source (e.g. a coding-QA check), not a better model.
- A simple Logistic Regression *beat* a tuned XGBoost model on this dataset (49.5% vs. 44.7% capture rate on validation). With ~2,100 training rows and a handful of strong binary "gap" signals (auth missing, referral missing, etc.), there isn't enough data for a more flexible model to find anything the simpler one misses — and the simpler model is also the easier one to explain to a reviewer.

3. What was built

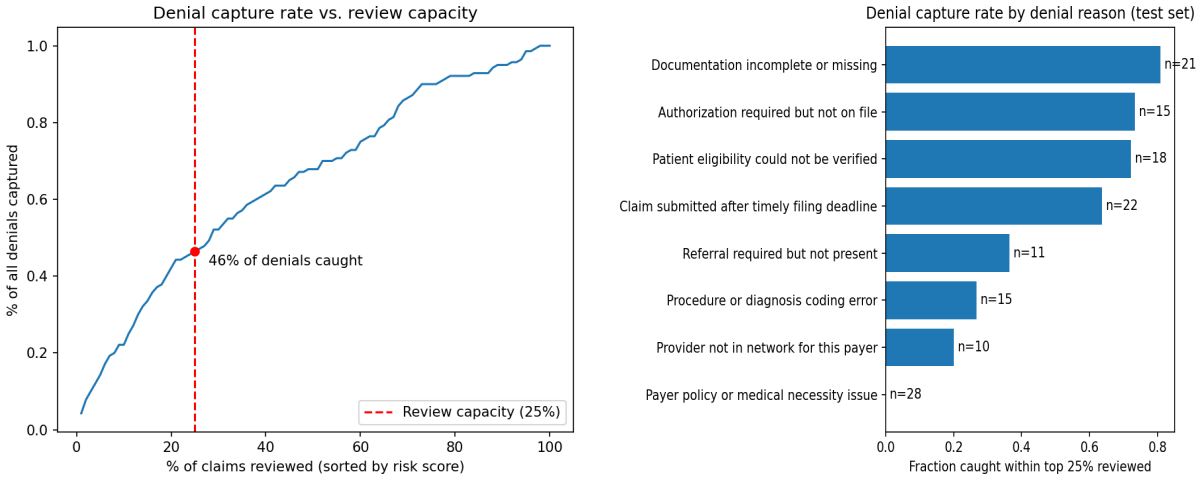
Feature engineering added a handful of "gap" flags that mirror how a reviewer actually reasons about a claim (prior-auth required-but-missing, referral required-but-missing, eligibility not verified, out-of-network, submitted >30 days late), a payment ratio, and a cyclical month-of-year encoding (the raw service_month column has non-overlapping years between history and current claims, so using it directly would teach the model to key off the literal year rather than seasonality).

Two candidates were compared on the validation split: Logistic Regression (baseline, class-balanced) and XGBoost (regularized: max_depth=3, min_child_weight=5, L2=2.0, early stopping). Logistic Regression won on capture rate at top 25% and was selected as the final model. Risk-tier thresholds (High ≥ 75th percentile of validation scores, Medium ≥ 50th percentile, Low below) were fit on the validation split specifically to avoid overfitting to train and to avoid leaking test-set information — High corresponds by construction to the top-25% review capacity.

4. Test-set metrics

Metric	Value
ROC-AUC	0.698
PR-AUC	0.516
Base denial rate (test)	26.0%
Capture rate @ top 25% reviewed	46.4%
Precision @ top 25% reviewed	48.1%

In plain terms: reviewing just the top quarter of claims by predicted risk catches nearly half of all claims that go on to be denied — roughly 1.8x better than reviewing a random quarter of claims would.



Left: denial capture rate vs. review capacity, with the 25% operating point marked. Right: capture rate broken out by denial reason (test set).

5. LLM explanations (Part 2)

For the top 10 highest-risk current claims, explain.py sends Gemini Flash a prompt containing only the claim's actual field values and its SHAP-derived top risk drivers, with explicit instructions not to invent facts, to name exactly one concrete action, and to caveat that this is a risk estimate. This sandbox has no network access to Google's API, so the example below was hand-drafted against the real prompt and real field values (two more examples, including a low-risk sanity check, are in outputs/example_explanations.md).

Prompt template (abridged): "You are helping a hospital billing analyst quickly triage a claim... Claim facts (use ONLY these): Payer, visit type, billed/expected amounts, days to submit, prior auth required/on file, network status, documentation flag, eligibility verified, referral required/on file. Model output: denial probability, top risk drivers. Write a 2-3 sentence explanation with exactly one recommended action, and note this is a risk estimate, not a guarantee."

Example output (CCLM-00082, 95% denial probability, BCBS Inpatient, $87,589.75 billed): "This inpatient claim is missing required documentation and the patient's eligibility hasn't been verified, both of which are strongly linked to denials in similar past claims. Recommended action: verify eligibility and attach the missing documentation before submitting. This is a risk estimate based on historical patterns, not a guarantee of denial."

6. Limitations and what I'd improve with more time

- Denials tied to medical necessity or coding accuracy are close to unpredictable from the fields available pre-submission — the model is silent there by data-availability, not by a fixable modeling gap. I'd want a coding-QA signal or payer-specific medical-necessity flag to close this.

- 3,200 historical rows is thin for gradient boosting; I'd revisit XGBoost vs. Logistic Regression with more data or synthetic augmentation before trusting the simpler model long-term.
- denial_probability is currently validated only as a *ranking* signal (capture rate), not a calibrated probability — I'd add a reliability curve before treating it as a literal probability in any downstream reporting.